

```
=====
==
源码文件 1: src/main/java/com/evolunteer/EVolunteerApplication.java (Java 源程序)
=====
```

```
==
package com.evolunteer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@EnableScheduling
@SpringBootApplication
public class EVolunteerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EVolunteerApplication.class, args);
    }
}
=====
```

```
==
源码文件 2: src/main/java/com/evolunteer/config/MyBatisPlusConfig.java (Java 源程序)
=====
```

```
==
package com.evolunteer.config;

import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.OptimisticLockerInnerInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
@MapperScan(value = "com.evolunteer.mapper")
public class MyBatisPlusConfig {

    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {

        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();

        interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.MYSQL));

        interceptor.addInnerInterceptor(new OptimisticLockerInnerInterceptor());

        return interceptor;
    }
}
=====
```

```
==
源码文件 3: src/main/java/com/evolunteer/config/WebSecurityConfig.java (Java 源程序)
```

```
=====
==
package com.evolunteer.config;

import com.evolunteer.service.impl.PasswordUpgradeServiceImpl;
import com.evolunteer.service.impl.UserDetailsServiceImpl;
import com.evolunteer.utils.LoginSuccessHandler;
import com.evolunteer.utils.SecurePasswordEncoder;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.authentication.builders.AuthenticationMa
nagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerA
dapter;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    UserDetailsServiceImpl userDetailsService;

    @Autowired
    PasswordUpgradeServiceImpl passwordUpgradeService;

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.userDetailsService(userDetailsService)
            .passwordEncoder(passwordEncoder())
            .userDetailsService(passwordUpgradeService);
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .formLogin()
            .loginPage("/admin/login.html")
            .loginProcessingUrl("/login")
            .usernameParameter("username")
            .passwordParameter("password")
            .successHandler(new LoginSuccessHandler());

        http
            .logout()
            .logoutUrl("/logout")
            .logoutSuccessUrl("/admin/login.html")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID");

        http
```

```

        .authorizeRequests()

        .antMatchers("/portal/js/**", "/portal/css/**", "/portal/images/**", "/portal/
fonts/**").permitAll()
        .antMatchers("/console/assets/**").permitAll()

        .antMatchers("/portal/**", "/portal/index.html", "/index.html", "/", "/inde
x").permitAll()

        .antMatchers("/admin/sign-up.html", "/admin/login.html").permitAll()

        .antMatchers("/loginRegisterController/**").permitAll()

        .antMatchers("/certificate/**").permitAll()

        .antMatchers("/file/download/**").permitAll()

        .antMatchers("/portal/site/**").permitAll()

        .antMatchers("/volunteer/**").hasRole("VOLUNTEER")
        .antMatchers("/checkin/in", "/checkin/out").hasRole("VOLUNTEER")

        .antMatchers("/checkin/**").hasRole("ORGANIZATION")

        .antMatchers("/activity/**").hasAnyRole("ADMIN", "ORGANIZATION")

        .antMatchers("/admin/**").hasRole("ADMIN")

        .antMatchers("/org/**", "/participate/**").hasRole("ORGANIZATION")
        .anyRequest().authenticated();

    http.csrf().disable();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new SecurePasswordEncoder();
}
}

```

```

=====
==
源码文件 4: src/main/java/com/evolunteer/controller/ActivityController.java (Java 源程序)
=====
==

```

```

package com.evolunteer.controller;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.evolunteer.entity.Activity;
import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.Participation;
import com.evolunteer.enums.AuditActionEnum;
import com.evolunteer.service.ActivityService;
import com.evolunteer.service.AuditLogService;

```

```
import com.evoluteer.service.MatchingService;
import com.evoluteer.service.OrganizationService;
import com.evoluteer.service.ParticipationService;
import com.evoluteer.utils.MatchCalculator;
import com.evoluteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import javax.servlet.http.HttpServletRequest;
import java.util.List;
import java.util.Map;

@Slf4j
@Controller
@RequestMapping(value = "/activity")
public class ActivityController {

    @Autowired
    ActivityService activityService;

    @Autowired
    OrganizationService organizationService;

    @Autowired
    ParticipationService participationService;

    @Autowired
    MatchingService matchingService;

    @Autowired
    AuditLogService auditLogService;

    @RequestMapping(value = "/findAuditActivityByNumAndState", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse findAuditActivityByNumAndState(@RequestParam(value = "page", required =
false) Integer page,
                                                    @RequestParam(value = "size", required =
false) Integer size,
                                                    @RequestParam(value = "keyword", require
d = false) String keyword,
                                                    Authentication authentication) {
        Integer num = currentOrganizationNum(authentication);
        return PageSupport.toResponse(activityService.pageActivityByOrganization(num, "0", key
word, page, size));
    }

    @RequestMapping(value = "/activityInfo", method = RequestMethod.GET)
```

```

@ResponseBody
public ApiResponse activityInfo(@RequestParam(value = "activityNum") Integer activityNum)
{
    List<Activity> activityInfo = activityService.activityInfo(activityNum);
    return ApiResponse.success().add("activityInfo", activityInfo);
}

@RequestMapping(value = "/auditActivityDelete", method = RequestMethod.POST)
@ResponseBody
public ApiResponse auditActivityDelete(@RequestParam(value = "activityNum") Integer activityNum,
                                       Authentication authentication) {
    activityService.deleteAct(activityNum);
    log.info("志愿者组织删除审核中的活动, 活动编号: {}", activityNum);
    return findAuditActivityByNumAndState(null, null, null, authentication);
}

@RequestMapping(value = "/notAuditActivityDelete", method = RequestMethod.POST)
@ResponseBody
public ApiResponse notAuditActivityDelete(@RequestParam(value = "activityNum") Integer activityNum,
                                          Authentication authentication) {
    activityService.deleteAct(activityNum);
    log.info("志愿者组织删除审核未通过的活动, 活动编号: {}", activityNum);
    return findNotAuditActivityByNumAndState(null, null, null, authentication);
}

@RequestMapping(value = "/findNotStartActivityByNumAndState", method = RequestMethod.GET)
@ResponseBody
public ApiResponse findNotStartActivityByNumAndState(@RequestParam(value = "page", required = false) Integer page,
                                                      @RequestParam(value = "size", required = false) Integer size,
                                                      @RequestParam(value = "keyword", required = false) String keyword,
                                                      Authentication authentication) {
    Integer num = currentOrganizationNum(authentication);
    return PageSupport.toResponse(activityService.pageActivityByOrganization(num, "1", keyword, page, size));
}

@RequestMapping(value = "/volRecruit", method = RequestMethod.GET)
@ResponseBody
public ApiResponse volRecruit(@RequestParam(value = "activityNum") Integer activityNum,
                              @RequestParam(value = "page", required = false) Integer page,
                              @RequestParam(value = "size", required = false) Integer size,
                              @RequestParam(value = "keyword", required = false) String keyword) {
    IPage<Participation> result = participationService.pageVolunteerList(activityNum, keyword, page, size);
    for (Participation participation : result.getRecords()) {
        MatchCalculator.MatchResult match = matchingService.matchForActivity(
            participation.getVolunteerNum(), activityNum);
        if (match != null) {
            participation.setMatchScore(match.getScore());
            participation.setMatchReasons(String.join("、", match.getReasons()));
        }
    }
}

```

```

        return PageSupport.toResponse(result);
    }

    @RequestMapping(value = "/findCarryActivityByNumAndState", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse findCarryActivityByNumAndState(@RequestParam(value = "page", required =
false) Integer page,
                                                    @RequestParam(value = "size", required =
false) Integer size,
                                                    @RequestParam(value = "keyword", require
d = false) String keyword,
                                                    Authentication authentication) {
        Integer num = currentOrganizationNum(authentication);
        return PageSupport.toResponse(activityService.pageActivityByOrganization(num, "2", key
word, page, size));
    }

    @RequestMapping(value = "/findEndActivityByNumAndState", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse findEndActivityByNumAndState(@RequestParam(value = "page", required = f
alse) Integer page,
                                                    @RequestParam(value = "size", required = f
alse) Integer size,
                                                    @RequestParam(value = "keyword", required
= false) String keyword,
                                                    Authentication authentication) {
        Integer num = currentOrganizationNum(authentication);
        return PageSupport.toResponse(activityService.pageActivityByOrganization(num, "3", key
word, page, size));
    }

    @RequestMapping(value = "/findNotAuditActivityByNumAndState", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse findNotAuditActivityByNumAndState(@RequestParam(value = "page", require
d = false) Integer page,
                                                    @RequestParam(value = "size", require
d = false) Integer size,
                                                    @RequestParam(value = "keyword", requ
ired = false) String keyword,
                                                    Authentication authentication) {
        Integer num = currentOrganizationNum(authentication);
        return PageSupport.toResponse(activityService.pageActivityByOrganization(num, "4", key
word, page, size));
    }

    @RequestMapping(value = "/startActivity", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse startActivity(@RequestParam(value = "activityNum") Integer activityNum,
                                    Authentication authentication, HttpServletRequest request
t) {
        return changeState(activityNum, "2", authentication, request);
    }

    @RequestMapping(value = "/finishActivity", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse finishActivity(@RequestParam(value = "activityNum") Integer activityNu
m,
                                    Authentication authentication, HttpServletRequest request
t) {
        return changeState(activityNum, "3", authentication, request);
    }

    private ApiResponse changeState(Integer activityNum, String targetState,
                                    Authentication authentication, HttpServletRequest request)
{

```

```

        User user = (User) authentication.getPrincipal();
        Map<Object, Object> result = activityService.changeActivityState(user.getUsername(), activityNum, targetState);
        String msg = (String) result.get("msg");
        Object ok = result.get("ok");
        if (ok == null || ((Number) ok).intValue() != 1) {
            log.warn("活动状态流转未完成, 活动编号: {}, 原因: {}", activityNum, msg);
            return ApiResponse.fail(msg);
        }
        auditLogService.record(user.getUsername(), "ROLE_ORGANIZATION", AuditActionEnum.ACTIVITY_STATE_CHANGE,
            "活动编号 " + activityNum, msg, request.getRemoteAddr());
        log.info("志愿活动状态流转, 活动编号: {}, 目标状态: {}", activityNum, targetState);
        return ApiResponse.success(msg);
    }

    private Integer currentOrganizationNum(Authentication authentication) {
        User user = (User) authentication.getPrincipal();
        return organizationService.getOrganizationNum(user.getUsername());
    }
}

```

=====
 ==
 源码文件 5: src/main/java/com/evolunteer/controller/AdminAccountController.java (Java 源程序)
 =====

```

package com.evolunteer.controller;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.Organization;
import com.evolunteer.entity.Volunteer;
import com.evolunteer.enums.AuditActionEnum;
import com.evolunteer.service.AccountAdminService;
import com.evolunteer.service.AuditLogService;
import com.evolunteer.service.ExportService;
import com.evolunteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.Map;

```

```

@Slf4j
@Controller
@RequestMapping(value = "/admin/account")
public class AdminAccountController {

    private static final Integer ENABLED = 1;

    private static final Integer DISABLED = 0;

    private static final DateTimeFormatter FILE_DATE_FORMAT = DateTimeFormatter.ofPattern("yyy
yMMdd");

    @Autowired
    AccountAdminService accountAdminService;

    @Autowired
    ExportService exportService;

    @Autowired
    AuditLogService auditLogService;

    @RequestMapping(value = "/organizations", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse pageOrganizations(@RequestParam(value = "page", required = false) Integer page,
                                         @RequestParam(value = "size", required = false) Integer size,
                                         @RequestParam(value = "keyword", required = false) String keyword) {
        IPage<Organization> organizations = accountAdminService.pageOrganizations(keyword, page, size);
        return PageSupport.toResponse(organizations);
    }

    @RequestMapping(value = "/volunteers", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse pageVolunteers(@RequestParam(value = "page", required = false) Integer page,
                                       @RequestParam(value = "size", required = false) Integer size,
                                       @RequestParam(value = "keyword", required = false) String keyword) {
        IPage<Volunteer> volunteers = accountAdminService.pageVolunteers(keyword, page, size);
        return PageSupport.toResponse(volunteers);
    }

    @RequestMapping(value = "/organization/create", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse createOrganization(@RequestParam(value = "organizationId") String organizationId,
                                         @RequestParam(value = "organizationName") String organizationName,
                                         @RequestParam(value = "password", required = false) String password,
                                         @RequestParam(value = "establishDate", required = false) String establishDate,
                                         @RequestParam(value = "description", required = false) String description,
                                         Authentication authentication,
                                         HttpServletRequest request) {

        User user = (User) authentication.getPrincipal();

```

```

        String loginId = user.getUsername();

        Map<Object, Object> result = accountAdminService.createOrganization(
            loginId, organizationId, organizationName, password, establishDate, descriptio
n);
        auditLogService.record(loginId, roleOf(authentication), AuditActionEnum.ACCOUNT_CREAT
E,
            organizationId, "开通组织账号: " + messageOf(result), request.getRemoteAddr());
        log.info("平台管理员开通志愿者组织账号, 组织账号: {}, 处理结果: {}", organizationId, result.ge
t("msg"));

        ApiResponse response = responseOf(result);
        addPassword(response, result);
        return response;
    }

    @RequestMapping(value = "/enabled", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse setEnabled(@RequestParam(value = "accountId") String accountId,
        @RequestParam(value = "enabled") Integer enabled,
        Authentication authentication,
        HttpServletRequest request) {

        if (!ENABLED.equals(enabled) && !DISABLED.equals(enabled)) {
            return ApiResponse.fail("账号状态参数不正确, 请刷新页面后重试");
        }

        User user = (User) authentication.getPrincipal();
        String loginId = user.getUsername();

        Map<Object, Object> result = accountAdminService.setEnabled(loginId, accountId, enable
d);
        auditLogService.record(loginId, roleOf(authentication), AuditActionEnum.ACCOUNT_ENABLE
D_CHANGE,
            accountId, "账号启停: " + messageOf(result), request.getRemoteAddr());
        log.info("平台管理员{}账号, 目标账号: {}, 处理结果: {}",
            ENABLED.equals(enabled) ? "启用" : "停用", accountId, result.get("msg"));
        return responseOf(result);
    }

    @RequestMapping(value = "/password/reset", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse resetPassword(@RequestParam(value = "accountId") String accountId,
        Authentication authentication,
        HttpServletRequest request) {

        User user = (User) authentication.getPrincipal();
        String loginId = user.getUsername();

        Map<Object, Object> result = accountAdminService.resetPassword(loginId, accountId);
        auditLogService.record(loginId, roleOf(authentication), AuditActionEnum.PASSWORD_RESE
T,
            accountId, "重置账号密码: " + messageOf(result), request.getRemoteAddr());
        log.info("平台管理员重置账号密码, 目标账号: {}, 处理结果: {}", accountId, result.get("msg"));

        ApiResponse response = responseOf(result);
    }

```

```
        addPassword(response, result);
        return response;
    }

    @RequestMapping(value = "/export/volunteers", method = RequestMethod.GET)
    public void exportVolunteers(@RequestParam(value = "keyword", required = false) String keyword,
        HttpServletResponse response) throws IOException {
        String fileName = "volunteers-" + LocalDate.now().format(FILE_DATE_FORMAT) + ".csv";
        writeCsv(response, fileName, exportService.exportVolunteers(keyword));
        log.info("平台管理员导出志愿者名册, 关键字: {}", keyword, fileName);
    }

    @RequestMapping(value = "/export/activities", method = RequestMethod.GET)
    public void exportActivities(@RequestParam(value = "keyword", required = false) String keyword,
        HttpServletResponse response) throws IOException {
        String fileName = "activities-" + LocalDate.now().format(FILE_DATE_FORMAT) + ".csv";
        writeCsv(response, fileName, exportService.exportActivities(keyword));
        log.info("平台管理员导出活动清单, 关键字: {}", keyword, fileName);
    }

    private void writeCsv(HttpServletResponse response, String fileName, String csv) throws IOException {
        response.setContentType("text/csv;charset=UTF-8");
        response.setCharacterEncoding("UTF-8");
        response.setHeader("Content-Disposition", "attachment;filename=" + fileName);
        response.getWriter().write(csv);
    }

    private ApiResponse responseOf(Map<Object, Object> result) {
        return isOk(result) ? ApiResponse.success(messageOf(result)) : ApiResponse.fail(messageOf(result));
    }

    private boolean isOk(Map<Object, Object> result) {
        Object ok = result == null ? null : result.get("ok");
        return ok != null && "1".equals(ok.toString());
    }

    private void addPassword(ApiResponse response, Map<Object, Object> result) {
        if (!isOk(result)) {
            return;
        }
        Object password = result.get("password");
        if (password != null) {
            response.add("password", password);
        }
    }

    private String messageOf(Map<Object, Object> result) {
        Object msg = result == null ? null : result.get("msg");
        return msg == null ? "操作处理失败, 请稍后重试" : msg.toString();
    }
}
```

```

        private String roleOf(Authentication authentication) {
            if (authentication == null || authentication.getAuthorities().isEmpty()) {
                return "ROLE_ADMIN";
            }
            return authentication.getAuthorities().iterator().next().getAuthority();
        }
    }
}

```

=====
 ==
 源码文件 6: src/main/java/com/evolunteer/controller/AdminAnnouncementController.java (Java 源程序)
 =====

```

==
package com.evolunteer.controller;

import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.PolicyFile;
import com.evolunteer.service.AnnouncementService;
import com.evolunteer.service.ShowService;
import com.evolunteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.multipart.MultipartFile;

import javax.servlet.http.HttpServletRequest;
import java.util.List;
import java.util.Map;

@Slf4j
@Controller
@RequestMapping(value = "/admin")
public class AdminAnnouncementController {

    @Autowired
    AnnouncementService announcementService;

    @Autowired
    ShowService showService;

    @RequestMapping(value = "/announcement/list", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse list(@RequestParam(value = "page", required = false) Integer page,
                           @RequestParam(value = "size", required = false) Integer size,
                           @RequestParam(value = "keyword", required = false) String keyword)
{

```

```

        return PageSupport.toResponse(announcementService.page(page, size, keyword));
    }

    @RequestMapping(value = "/announcement/save", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse save(@RequestParam(value = "policyannouncementNum", required = false) Integer policyannouncementNum,
                           @RequestParam(value = "name", required = false) String name,
                           @RequestParam(value = "detail", required = false) String detail,
                           Authentication authentication) {

        User user = (User) authentication.getPrincipal();
        Map<Object, Object> result = announcementService.save(policyannouncementNum, name, detail, user.getUsername());
        String msg = (String) result.get("msg");
        if (!isSuccess(result)) {
            log.warn("通知公告保存未成功, 公告编号: {}, 原因: {}", policyannouncementNum, msg);
            return ApiResponse.fail(msg);
        }
        return ApiResponse.success(msg).add("policyannouncementNum", result.get("policyannouncementNum"));
    }

    @RequestMapping(value = "/announcement/delete", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse delete(@RequestParam(value = "policyannouncementNum") Integer policyannouncementNum) {

        Map<Object, Object> result = announcementService.delete(policyannouncementNum);
        String msg = (String) result.get("msg");
        if (!isSuccess(result)) {
            return ApiResponse.fail(msg);
        }
        return ApiResponse.success(msg);
    }

    @RequestMapping(value = "/announcement/file", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse uploadFile(@RequestParam(value = "policyannouncementNum") Integer policyannouncementNum,
                                  @RequestParam(value = "file") MultipartFile file) {
        try {
            PolicyFile policyFile = announcementService.uploadFile(policyannouncementNum, file);
            log.info("平台管理员上传公告附件, 公告编号: {}, 附件编号: {}",
                    policyannouncementNum, policyFile.getPolicyfileNum());
            return ApiResponse.success("附件已上传");
        } catch (IllegalArgumentException | IllegalStateException e) {
            return ApiResponse.fail(e.getMessage());
        }
    }

    @RequestMapping(value = "/announcement/file/delete", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse deleteFile(@RequestParam(value = "policyfileNum") Integer policyfileNum) {
        if (!announcementService.deleteFile(policyfileNum)) {

```

```

        return ApiResponse.fail("该附件不存在或已被删除");
    }
    return ApiResponse.success("附件已删除");
}

@RequestMapping(value = "/announcement/files", method = RequestMethod.GET)
@ResponseBody
public ApiResponse files(@RequestParam(value = "policyannouncementNum") Integer policyannouncementNum) {
    List<PolicyFile> files = announcementService.listFiles(policyannouncementNum);
    return ApiResponse.success().add("files", files);
}

@RequestMapping(value = "/review/shows", method = RequestMethod.GET)
@ResponseBody
public ApiResponse shows(@RequestParam(value = "page", required = false) Integer page,
                        @RequestParam(value = "size", required = false) Integer size,
                        @RequestParam(value = "keyword", required = false) String keyword
d) {
    return PageSupport.toResponse(showService.pageAdmin(page, size, keyword));
}

@RequestMapping(value = "/review/show/delete", method = RequestMethod.POST)
@ResponseBody
public ApiResponse deleteShow(@RequestParam(value = "showNum") Integer showNum,
                             Authentication authentication,
                             HttpServletRequest request) {

    User user = (User) authentication.getPrincipal();
    Map<Object, Object> result = showService.deleteShow(showNum, user.getUsername(), request.getRemoteAddr());
    String msg = (String) result.get("msg");
    if (!isSuccess(result)) {
        return ApiResponse.fail(msg);
    }
    return ApiResponse.success(msg);
}

private boolean isSuccess(Map<Object, Object> result) {
    Object ok = result.get("ok");
    return ok != null && ((Number) ok).intValue() == 1;
}
}

```

```

=====
==
源码文件 7: src/main/java/com/evolunteer/controller/AdminAuditController.java (Java 源程序)
=====
==

```

```

package com.evolunteer.controller;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.AuditActionView;
import com.evolunteer.entity.AuditLog;

```

```

import com.evoluteer.enums.AuditActionEnum;
import com.evoluteer.service.AuditLogService;
import com.evoluteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.ArrayList;
import java.util.List;

@Slf4j
@Controller
@RequestMapping(value = "/admin/audit")
public class AdminAuditController {

    @Autowired
    AuditLogService auditLogService;

    @RequestMapping(value = "/list", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse list(@RequestParam(value = "page", required = false) Integer page,
                           @RequestParam(value = "size", required = false) Integer size,
                           @RequestParam(value = "action", required = false) String action,
                           @RequestParam(value = "operator", required = false) String operator) {

        IPage<AuditLog> auditLogs = auditLogService.page(action, operator, page, size);
        log.info("平台管理员查询操作审计日志, 操作类型: {}, 操作账号: {}", action, operator);
        return PageSupport.toResponse(auditLogs).add("actions", actionOptions());
    }

    private List<AuditActionView> actionOptions() {
        AuditActionEnum[] actions = AuditActionEnum.values();
        List<AuditActionView> options = new ArrayList<>(actions.length);
        for (AuditActionEnum action : actions) {
            options.add(new AuditActionView(action.getActionCode(), action.getActionName()));
        }
        return options;
    }
}

```

```

=====
==
源码文件 8: src/main/java/com/evoluteer/controller/AdminController.java (Java 源程序)
=====
==

```

```

package com.evoluteer.controller;

import com.evoluteer.entity.Activity;
import com.evoluteer.entity.ApiResponse;

```

```

import com.evoluteer.enums.ActivityRejectReasonEnum;
import com.evoluteer.enums.AuditActionEnum;
import com.evoluteer.service.ActivityService;
import com.evoluteer.service.AuditLogService;
import com.evoluteer.service.CheckInService;
import com.evoluteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import javax.servlet.http.HttpServletRequest;
import java.util.HashMap;
import java.util.Map;

@Slf4j
@Controller
@RequestMapping(value = "/admin")
public class AdminController {

    @Autowired
    ActivityService activityService;

    @Autowired
    CheckInService checkInService;

    @Autowired
    AuditLogService auditLogService;

    @RequestMapping(value = "/findAuditActivityByState", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse findAuditActivityByState(@RequestParam(value = "page", required = false) Integer page,
                                                @RequestParam(value = "size", required = false) Integer size,
                                                @RequestParam(value = "keyword", required = false) String keyword) {
        return PageSupport.toResponse(activityService.pageActivityByState("0", keyword, page, size));
    }

    @RequestMapping(value = "/passAct", method = RequestMethod.POST)
    @ResponseBody
    public String passAct(@RequestParam(value = "actName") String actName,
                          @RequestParam(value = "statue") String statue,
                          @RequestParam(value = "reasonCode", required = false) String reasonCode,
                          @RequestParam(value = "remark", required = false) String remark,
                          Authentication authentication, HttpServletRequest request) {

```

```

        Activity activity = activityService.selectAct(actName);
        if (activity == null) {
            log.warn("平台管理员审核的活动不存在, 活动名称: {}", actName);
            return "该志愿活动不存在, 请刷新页面后重试";
        }

        if (!"1".equals(statue) && ActivityRejectReasonEnum.of(reasonCode) == null) {
            log.warn("平台管理员审核未通过但未给出原因, 活动编号: {}", activity.getActivityNum());
            return "请选择审核不通过的原因";
        }

        User user = (User) authentication.getPrincipal();

        Map<Object, Object> map = new HashMap<>();
        map.put("loginId", user.getUsername());
        map.put("activityNum", activity.getActivityNum());
        map.put("isPass", statue);
        map.put("reasonCode", reasonCode);
        map.put("remark", remark);
        activityService.passActByAdminIdWithStatue(map);

        String msg = (String) map.get("msg");
        auditLogService.record(user.getUsername(), "ROLE_ADMIN", AuditActionEnum.ACTIVITY_AUDIT,
T,
            "活动编号 " + activity.getActivityNum(), msg + ", 原因: " + reasonCode, request.
getRemoteAddr());
        log.info("平台管理员完成活动审核, 活动编号: {}, 审核结果: {}, 原因: {}", activity.getActivityNum(), statue, reasonCode);
        return msg;
    }

    @RequestMapping(value = "/review/manualHours", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse manualHours(@RequestParam(value = "page", required = false) Integer page,
e,
        @RequestParam(value = "size", required = false) Integer size,
e,
        @RequestParam(value = "state", required = false) String state,
te,
        @RequestParam(value = "keyword", required = false) String keyword) {
        return PageSupport.toResponse(checkInService.pageManualCheckins(state, keyword, page, size));
    }

    @RequestMapping(value = "/review/anomalies", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse anomalies(@RequestParam(value = "page", required = false) Integer page,
        @RequestParam(value = "size", required = false) Integer size,
        @RequestParam(value = "keyword", required = false) String keyword) {
word) {
        return PageSupport.toResponse(checkInService.pageAnomalies(keyword, page, size));
    }

    @RequestMapping(value = "/review/manualHours/check", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse checkManualHours(@RequestParam(value = "checkinNum") Integer checkinNum,
m,
        @RequestParam(value = "isPass") Integer isPass,
        @RequestParam(value = "remark", required = false) String remark,

```

```

Authentication authentication, HttpServletRequest request) {
    String loginId = ((User) authentication.getPrincipal()).getUsername();
    Map<Object, Object> result = checkInService.reviewByAdmin(loginId, checkinNum, isPass,
    remark);
    String msg = (String) result.get("msg");
    Object ok = result.get("ok");
    if (ok == null || ((Number) ok).intValue() != 1) {
        log.warn("补录服务时长复核未完成, 签到编号: {}", 原因: {}", checkinNum, msg);
        return ApiResponse.fail(msg);
    }
    auditLogService.record(loginId, "ROLE_ADMIN", AuditActionEnum.MANUAL_HOUR_REVIEW,
        "签到编号 " + checkinNum, msg, request.getRemoteAddr());
    log.info("平台管理员完成服务时长裁定, 签到编号: {}, 结果: {}", checkinNum, msg);
    return ApiResponse.success(msg);
}
}

```

```

=====
==
源码文件 9: src/main/java/com/evolunteer/controller/CertificateController.java (Java 源程序)
=====
==

```

```

package com.evolunteer.controller;

import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.ServiceCertificate;
import com.evolunteer.service.CertificateService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

@Slf4j
@Controller
@RequestMapping(value = "/certificate")
public class CertificateController {

    @Autowired
    CertificateService certificateService;

    @RequestMapping(value = "/verify", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse verify(@RequestParam(value = "volunteerId") String volunteerId,
        @RequestParam(value = "code") String code) {

        ServiceCertificate certificate = certificateService.verifyCertificate(volunteerId, code);

        if (certificate == null) {
            log.warn("志愿服务证明校验失败, 未找到志愿者: {}", volunteerId);
            return ApiResponse.fail("未找到该志愿者编号对应的服务记录");
        }
    }
}

```

```

    }
    if (!certificate.isValid()) {
        log.warn("志愿服务证明校验码不正确, 志愿者编号: {}", volunteerId);
        return ApiResponse.fail("校验码不正确, 该证明可能被篡改或不存在")
            .add("verification", certificate);
    }
    log.info("志愿服务证明校验通过, 志愿者编号: {}", volunteerId);
    return ApiResponse.success("校验通过").add("verification", certificate);
}
}

```

```

=====
==
源码文件 10: src/main/java/com/evolunteer/controller/CheckInController.java (Java 源程序)
=====
==

```

```

package com.evolunteer.controller;

import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.CheckinCodeView;
import com.evolunteer.entity.Volunteer;
import com.evolunteer.enums.AuditActionEnum;
import com.evolunteer.service.AuditLogService;
import com.evolunteer.service.CheckInService;
import com.evolunteer.service.VolunteerService;
import com.evolunteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import javax.servlet.http.HttpServletRequest;
import java.util.Map;

@Slf4j
@Controller
@RequestMapping(value = "/checkin")
public class CheckInController {

    @Autowired
    CheckInService checkInService;

    @Autowired
    VolunteerService volunteerService;

    @Autowired
    AuditLogService auditLogService;

```

```

@RequestMapping(value = "/in", method = RequestMethod.POST)
@ResponseBody
public ApiResponse checkin(@RequestParam(value = "activityNum") Integer activityNum,
                           @RequestParam(value = "code", required = false) String code,
                           @RequestParam(value = "latitude", required = false) String latitude,
                           @RequestParam(value = "longitude", required = false) String longitude,
                           Authentication authentication) {

    Volunteer volunteer = currentVolunteer(authentication);
    if (volunteer == null) {
        return ApiResponse.fail("未找到志愿者信息，请重新登录后再试");
    }

    Map<Object, Object> result = checkInService.checkin(volunteer.getVolunteerNum(), activityNum, code,
        parseCoordinate(latitude), parseCoordinate(longitude));
    String msg = (String) result.get("msg");
    if (!isSuccess(result)) {
        log.warn("志愿者签到未成功，志愿者编号: {}, 活动编号: {}, 原因: {}", volunteer.getVolunteerId(), activityNum, msg);
        return ApiResponse.fail(msg);
    }
    log.info("志愿者签到成功，志愿者编号: {}, 活动编号: {}, 轨迹标记: {}", volunteer.getVolunteerId(), activityNum,
        result.get("flag"));
    return ApiResponse.success(msg).add("flag", result.get("flag"));
}

@RequestMapping(value = "/out", method = RequestMethod.POST)
@ResponseBody
public ApiResponse checkout(@RequestParam(value = "activityNum") Integer activityNum,
                            @RequestParam(value = "latitude", required = false) String latitude,
                            @RequestParam(value = "longitude", required = false) String longitude,
                            Authentication authentication) {

    Volunteer volunteer = currentVolunteer(authentication);
    if (volunteer == null) {
        return ApiResponse.fail("未找到志愿者信息，请重新登录后再试");
    }

    Map<Object, Object> result = checkInService.checkout(volunteer.getVolunteerNum(), activityNum,
        parseCoordinate(latitude), parseCoordinate(longitude));
    String msg = (String) result.get("msg");
    if (!isSuccess(result)) {
        return ApiResponse.fail(msg);
    }
    log.info("志愿者签退成功，志愿者编号: {}, 活动编号: {}, 服务时长: {} 小时", volunteer.getVolunteerId(), activityNum,
        result.get("duration"));
    return ApiResponse.success(msg).add("duration", result.get("duration"));
}

@RequestMapping(value = "/reviewList", method = RequestMethod.GET)

```

```

@ResponseBody
public ApiResponse reviewList(@RequestParam(value = "activityNum") Integer activityNum,
                              @RequestParam(value = "page", required = false) Integer page,
                              @RequestParam(value = "size", required = false) Integer size,
                              @RequestParam(value = "state", required = false) String state,
                              @RequestParam(value = "source", required = false) String source) {
    return PageSupport.toResponse(checkInService.pageCheckinRecords(activityNum, state, source, page, size));
}

@RequestMapping(value = "/review", method = RequestMethod.POST)
@ResponseBody
public ApiResponse review(@RequestParam(value = "checkinNum") Integer checkinNum,
                          @RequestParam(value = "isPass") Integer isPass,
                          @RequestParam(value = "remark", required = false) String remark,
                          HttpServletRequest request, Authentication auth) {

    Map<Object, Object> result = checkInService.reviewCheckin(checkinNum, isPass, remark);
    String msg = (String) result.get("msg");
    if (!isSuccess(result)) {
        log.warn("服务时长复核未完成, 签到编号: {}, 原因: {}", checkinNum, msg);
        return ApiResponse.fail(msg);
    }
    auditLogService.record(operator(auth), "ROLE_ORGANIZATION", AuditActionEnum.CHECKIN_REVIEW,
                          "签到编号 " + checkinNum, msg, request.getRemoteAddr());
    log.info("志愿者组织完成服务时长复核, 签到编号: {}, 结果: {}", checkinNum, msg);
    return ApiResponse.success(msg);
}

@RequestMapping(value = "/manual", method = RequestMethod.POST)
@ResponseBody
public ApiResponse manual(@RequestParam(value = "participateNum") Integer participateNum,
                          @RequestParam(value = "beginTime") String beginTime,
                          @RequestParam(value = "endTime") String endTime,
                          @RequestParam(value = "remark", required = false) String remark,
                          HttpServletRequest request, Authentication auth) {

    String loginId = operator(auth);
    Map<Object, Object> result = checkInService.recordServiceHours(loginId, participateNum,
                          beginTime, endTime, remark);
    String msg = (String) result.get("msg");
    if (!isSuccess(result)) {
        log.warn("服务时长补录未完成, 报名编号: {}, 原因: {}", participateNum, msg);
        return ApiResponse.fail(msg);
    }
    auditLogService.record(loginId, "ROLE_ORGANIZATION", AuditActionEnum.CHECKIN_REVIEW,
                          "报名编号 " + participateNum, "补录服务时长: " + msg, request.getRemoteAddr());
    return ApiResponse.success(msg);
}

@RequestMapping(value = "/code", method = RequestMethod.GET)

```

```

    @ResponseBody
    public ApiResponse getCheckinCode(@RequestParam(value = "activityNum") Integer activityNum) {
        CheckinCodeView codeInfo = checkInService.currentCheckinCode(activityNum);
        if (codeInfo == null) {
            return ApiResponse.fail("未找到对应的志愿活动, 无法获取签到码");
        }
        return ApiResponse.success()
            .add("checkinCode", codeInfo.getCheckinCode())
            .add("expiresInSeconds", codeInfo.getExpiresInSeconds());
    }

    @RequestMapping(value = "/confirmSheet", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse confirmSheet(@RequestParam(value = "checkinNum") Integer checkinNum,
        @RequestParam(value = "objectName") String objectName,
        @RequestParam(value = "objectPhone") String objectPhone,
        HttpServletRequest request, Authentication authentication)
    {
        String loginId = operator(authentication);
        Map<Object, Object> result = checkInService.issueConfirmSheet(loginId, checkinNum, objectName, objectPhone);
        String msg = (String) result.get("msg");
        if (!isSuccess(result)) {
            log.warn("生成服务确认单未完成, 签到编号: {}, 原因: {}", checkinNum, msg);
            return ApiResponse.fail(msg);
        }
        auditLogService.record(loginId, "ROLE_ORGANIZATION", AuditActionEnum.SERVICE_CONFIRM_SHEET,
            "签到编号 " + checkinNum, "生成服务确认单: " + objectName, request.getRemoteAddr());
        log.info("志愿者组织生成服务确认单, 签到编号: {}, 服务对象: {}", checkinNum, objectName);
        return ApiResponse.success(msg).add("confirmCode", result.get("confirmCode"));
    }

    @RequestMapping(value = "/code", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse refreshCheckinCode(@RequestParam(value = "activityNum") Integer activityNum) {
        String checkinCode = checkInService.refreshCheckinCode(activityNum);
        if (checkinCode == null) {
            return ApiResponse.fail("未找到对应的志愿活动, 无法生成签到码");
        }
        CheckinCodeView codeInfo = checkInService.currentCheckinCode(activityNum);
        log.info("志愿者组织更换签到密钥并重新生成现场签到码, 活动编号: {}", activityNum);
        return ApiResponse.success()
            .add("checkinCode", checkinCode)
            .add("expiresInSeconds", codeInfo == null ? null : codeInfo.getExpiresInSeconds());
    }

    private Volunteer currentVolunteer(Authentication authentication) {
        User user = (User) authentication.getPrincipal();
        return volunteerService.getByLoginId(user.getUsername());
    }

```

```

private String operator(Authentication authentication) {
    return ((User) authentication.getPrincipal()).getUsername();
}

private Double parseCoordinate(String value) {
    if (value == null || value.trim().isEmpty()) {
        return null;
    }
    try {
        return Double.valueOf(value.trim());
    } catch (NumberFormatException e) {
        return null;
    }
}

private boolean isSuccess(Map<Object, Object> result) {
    Object ok = result.get("ok");
    return ok != null && ((Number) ok).intValue() == 1;
}
}

```

```
=====
```

```
==
```

```
源码文件 11: src/main/java/com/evolunteer/controller/FileController.java (Java 源程序)
```

```
=====
```

```
==
```

```

package com.evolunteer.controller;

import com.evolunteer.service.FileStorageService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.Resource;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.ResponseBody;

@Slf4j
@Controller
@RequestMapping(value = "/file")
public class FileController {

    @Autowired
    FileStorageService fileStorageService;

    @RequestMapping(value = "/download/{storedName}", method = RequestMethod.GET)
    @ResponseBody
    public ResponseEntity<Resource> download(@PathVariable(value = "storedName") String stored
Name) {

```

```

        Resource resource;
        try {
            resource = fileStorageService.load(storedName);
        } catch (IllegalArgumentException e) {
            log.warn("文件下载失败, 存储名: {}", 原因: {}", storedName, e.getMessage());
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"" + storedName
e + "\"")
            .contentType(MediaType.APPLICATION_OCTET_STREAM)
            .body(resource);
    }
}

```

```

=====
==
源码文件 12: src/main/java/com/evolunteer/controller/LoginRegisterController.java (Java 源程序)
=====
==

```

```

package com.evolunteer.controller;

import com.evolunteer.service.VolunteerService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

@Slf4j
@Controller
@RequestMapping(value = "/loginRegisterController")
public class LoginRegisterController {

    @Autowired
    VolunteerService volunteerService;

    @RequestMapping(value = "/checkSignUpTel", method = RequestMethod.GET)
    @ResponseBody
    public boolean checkSignUpTel(@RequestParam(value = "signUpTel") String signUpTel) {
        return volunteerService.checkSignUpTel(signUpTel) <= 0;
    }

    @RequestMapping(value = "/signUp", method = RequestMethod.POST)
    @ResponseBody
    public String signUp(@RequestParam(value = "signUpName") String signUpName,
        @RequestParam(value = "signUpTel") String signUpTel,
        @RequestParam(value = "signUpPassword") String signUpPassword) {
        String volunteerId = volunteerService.signUp(signUpName, signUpTel, signUpPassword);
        if (volunteerId == null || volunteerId.isEmpty()) {

```

```

        log.warn("志愿者注册失败, 手机号已被注册: {}", signUpTel);
        return "该手机号已被注册, 请更换手机号后再试";
    }
    log.info("志愿者注册成功, 志愿者编号: {}", volunteerId);
    return volunteerId;
}
}

```

=====
 ==
 源码文件 13: src/main/java/com/evolunteer/controller/OrgContentController.java (Java 源程序)
 =====

```

==
package com.evolunteer.controller;

import com.evolunteer.entity.ActivityAnnouncement;
import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.Training;
import com.evolunteer.service.ActivityAnnouncementService;
import com.evolunteer.service.TrainingService;
import com.evolunteer.utils.DateTimeParser;
import com.evolunteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.Date;

@Slf4j
@Controller
@RequestMapping(value = "/org/content")
public class OrgContentController {

    private static final String MESSAGE_TIME_INVALID = "时间格式不正确, 请按年-月-日 时:分填写";

    @Autowired
    TrainingService trainingService;

    @Autowired
    ActivityAnnouncementService activityAnnouncementService;

    @RequestMapping(value = "/trainings", method = RequestMethod.GET)
    @ResponseBody
    public ApiResponse trainings(@RequestParam(value = "activityNum", required = false) Integer activityNum,
                                @RequestParam(value = "page", required = false) Integer page,
                                @RequestParam(value = "size", required = false) Integer size,

```

```

        Authentication authentication) {
    return PageSupport.toResponse(
        trainingService.pageByActivity(loginIdOf(authentication), activityNum, page, size));
    }

    @RequestMapping(value = "/training/save", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse saveTraining(@RequestParam(value = "trainingNum", required = false) Integer trainingNum,
                                   @RequestParam(value = "activityNum") Integer activityNum,
                                   @RequestParam(value = "name") String name,
                                   @RequestParam(value = "detail", required = false) String detail,
                                   @RequestParam(value = "beginTime") String beginTime,
                                   @RequestParam(value = "endTime") String endTime,
                                   @RequestParam(value = "location", required = false) String location,
                                   @RequestParam(value = "checkin", required = false) String checkin,
                                   Authentication authentication) {

        Date begin;
        Date end;
        try {
            begin = DateTimeParser.parse(beginTime);
            end = DateTimeParser.parse(endTime);
        } catch (IllegalArgumentException e) {
            log.warn("培训安排时间格式不正确, 开始时间: {}, 结束时间: {}", beginTime, endTime);
            return ApiResponse.fail(MESSAGE_TIME_INVALID);
        }

        Training training = new Training();
        training.setTrainingNum(trainingNum);
        training.setActivityNum(activityNum);
        training.setTrainingName(name);
        training.setTrainingDetail(detail);
        training.setTrainingBeginTime(begin);
        training.setTrainingEndTime(end);
        training.setTrainingLocation(location);
        training.setTrainingCheckin(checkin);

        String message = trainingService.saveTraining(loginIdOf(authentication), training);
        return respond(message, TrainingService.MESSAGE_SAVE_SUCCESS);
    }

    @RequestMapping(value = "/training/delete", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse deleteTraining(@RequestParam(value = "trainingNum") Integer trainingNum,
                                     Authentication authentication) {
        String message = trainingService.deleteTraining(loginIdOf(authentication), trainingNum);
        return respond(message, TrainingService.MESSAGE_DELETE_SUCCESS);
    }

    @RequestMapping(value = "/training/situations", method = RequestMethod.GET)

```

[illegible]

```

        ActivityAnnouncement announcement = new ActivityAnnouncement();
        announcement.setActannouncementNum(announcementNum);
        announcement.setActivityNum(activityNum);
        announcement.setActannouncementName(name);
        announcement.setActannouncementDetail(detail);

        String message = activityAnnouncementService.saveAnnouncement(loginIdOf(authenticatio
n), announcement);
        return respond(message, ActivityAnnouncementService.MESSAGE_SAVE_SUCCESS);
    }

    @RequestMapping(value = "/announcement/delete", method = RequestMethod.POST)
    @ResponseBody
    public ApiResponse deleteAnnouncement(@RequestParam(value = "announcementNum") Integer ann
ouncementNum,
                                         Authentication authentication) {
        String message = activityAnnouncementService.deleteAnnouncement(
            loginIdOf(authentication), announcementNum);
        return respond(message, ActivityAnnouncementService.MESSAGE_DELETE_SUCCESS);
    }

    private String loginIdOf(Authentication authentication) {
        User user = (User) authentication.getPrincipal();
        return user.getUsername();
    }

    private ApiResponse respond(String message, String successMessage) {
        return successMessage.equals(message) ? ApiResponse.success(message) : ApiResponse.fai
l(message);
    }
}

```

```

=====
==
源码文件 14: src/main/java/com/evolunteer/controller/OrganizationController.java (Java 源程序)
=====
==

```

```

package com.evolunteer.controller;

import com.evolunteer.entity.Activity;
import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.Organization;
import com.evolunteer.enums.AuditActionEnum;
import com.evolunteer.service.ActivityService;
import com.evolunteer.service.AuditLogService;
import com.evolunteer.service.CheckInService;
import com.evolunteer.service.NotificationService;
import com.evolunteer.service.OrgExportService;
import com.evolunteer.service.OrganizationService;
import com.evolunteer.utils.DateTimeParser;
import com.evolunteer.utils.PageSupport;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.User;

```

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;

import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@Slf4j
@Controller
@RequestMapping(value = "/org")
public class OrganizationController {

    @Autowired
    OrganizationService organizationService;

    @Autowired
    ActivityService activityService;

    @Autowired
    CheckInService checkInService;

    @Autowired
    NotificationService notificationService;

    @Autowired
    OrgExportService orgExportService;

    @Autowired
    AuditLogService auditLogService;

    @RequestMapping(value = "/upact", method = RequestMethod.POST)
    @ResponseBody
    public String upAct(@RequestParam(value = "name") String name,
                       @RequestParam(value = "content") String content,
                       @RequestParam(value = "startTime") String startTime,
                       @RequestParam(value = "endTime") String endTime,
                       @RequestParam(value = "location") String location,
                       @RequestParam(value = "affair") String affair,
                       @RequestParam(value = "need") String need,
                       @RequestParam(value = "sign_ddl") String sign_ddl,
                       @RequestParam(value = "tags", required = false) String tags,
                       @RequestParam(value = "latitude", required = false) String latitude,
```

```
        @RequestParam(value = "longitude", required = false) String longitude,
        @RequestParam(value = "radius", required = false) String radius,
        Authentication authentication) {

    Date start;
    Date end;
    Date signDdl;
    try {
        start = DateTimeParser.parse(startTime);
        end = DateTimeParser.parse(endTime);
        signDdl = DateTimeParser.parse(sign_ddl);
    } catch (IllegalArgumentException e) {
        log.warn("志愿活动申报时间格式不正确, 开始时间: {}, 结束时间: {}, 报名截止时间: {}", startTim
e, endTime, sign_ddl);
        return "活动时间格式不正确, 请按年-月-日 时:分重新填写";
    }

    Integer needPeople;
    try {
        needPeople = Integer.valueOf(need.trim());
        if (needPeople <= 0) {
            throw new NumberFormatException("招募人数必须大于 0");
        }
    } catch (RuntimeException e) {
        log.warn("志愿活动申报的招募人数格式不正确: {}", need);
        return "招募人数必须为大于 0 的数字, 请重新填写";
    }

    BigDecimal activityLatitude;
    BigDecimal activityLongitude;
    Integer activityRadius;
    try {
        activityLatitude = parseDecimal(latitude, 90, "活动地点纬度");
        activityLongitude = parseDecimal(longitude, 180, "活动地点经度");
        activityRadius = parseRadius(radius);
    } catch (IllegalArgumentException e) {
        log.warn("志愿活动申报的签到围栏参数不正确: {}", e.getMessage());
        return e.getMessage();
    }

    User user = (User) authentication.getPrincipal();

    Map<Object, Object> map = new HashMap<>();
    map.put("loginId", user.getUsername());
    map.put("activityName", name);
    map.put("activityDetail", content);
    map.put("activityBegintime", start);
    map.put("activityEndtime", end);
    map.put("activityLocation", location);
    map.put("activityNeedpeople", needPeople);
    map.put("activityNotice", affair);
```

```

        map.put("activitySignddl", signDdl);
        map.put("activitySkillNames", tags);
        map.put("activityLatitude", activityLatitude);
        map.put("activityLongitude", activityLongitude);
        map.put("activityRadius", activityRadius);
        organizationService.upAct(map);
        log.info("志愿者组织申报志愿活动, 活动名称: {}", name);
        return (String) map.get("msg");
    }

    @RequestMapping(value = "/reviseAct", method = RequestMethod.POST)
    @ResponseBody
    public String reviseAct(@RequestParam(value = "activityNum") Integer activityNum,
                           @RequestParam(value = "name") String name,
                           @RequestParam(value = "content") String content,
                           @RequestParam(value = "startTime") String startTime,
                           @RequestParam(value = "endTime") String endTime,
                           @RequestParam(value = "location") String location,
                           @RequestParam(value = "affair") String affair,
                           @RequestParam(value = "need") String need,
                           @RequestParam(value = "sign_ddl") String sign_ddl,
                           @RequestParam(value = "tags", required = false) String tags,
                           @RequestParam(value = "latitude", required = false) String latitude,
                           @RequestParam(value = "longitude", required = false) String longitude,
                           @RequestParam(value = "radius", required = false) String radius,
                           Authentication authentication, HttpServletRequest request) {

        Date start;
        Date end;
        Date signDdl;
        Integer needPeople;
        BigDecimal activityLatitude;
        BigDecimal activityLongitude;
        Integer activityRadius;
        try {
            start = DateTimeParser.parse(startTime);
            end = DateTimeParser.parse(endTime);
            signDdl = DateTimeParser.parse(sign_ddl);
            needPeople = Integer.valueOf(need.trim());
            if (needPeople <= 0) {
                throw new NumberFormatException("招募人数必须大于 0");
            }
            activityLatitude = parseDecimal(latitude, 90, "活动地点纬度");
            activityLongitude = parseDecimal(longitude, 180, "活动地点经度");
            activityRadius = parseRadius(radius);
        } catch (IllegalArgumentException e) {
            log.warn("志愿活动重新申报参数不正确: {}", e.getMessage());
            return e.getMessage();
        } catch (RuntimeException e) {
            log.warn("志愿活动重新申报的招募人数格式不正确: {}", need);
        }
    }

```

```
    }
    navigator.geolocation.getCurrentPosition(function (position) {
        $("#profile_latitude").val(position.coords.latitude);
        $("#profile_longitude").val(position.coords.longitude);
    }, function () {
        alert("未能获取当前位置, 请检查浏览器定位权限后重试");
    }, {enableHighAccuracy: true, timeout: 10000, maximumAge: 0});
}

function saveProfile() {
    var skills = $(".skill-checkbox:checked").map(function () {
        return $(this).val();
    }).get().join(",");
    $.ajax({
        url: "/volunteer/profile/save",
        type: "POST",
        data: {
            skills: skills,
            latitude: $.trim($("#profile_latitude").val()),
            longitude: $.trim($("#profile_longitude").val())
        },
        success: function (result) {
            alert((result && result.msg) ? result.msg : "操作完成");
            if (result && result.code === 100) {
                loadProfile();
            }
        },
        error: function (result) {
            console.log(result);
            alert("网络异常, 请稍后重试");
        }
    });
}

function changePassword() {
    var oldPassword = $("#old_password").val();
    var newPassword = $("#new_password").val();
    var confirmPassword = $("#confirm_password").val();
    if (!oldPassword) {
        alert("请输入原密码");
        return;
    }
    if (!newPassword || newPassword.length < 8 || newPassword.length > 20) {
        alert("新密码长度需为 8~20 位");
        return;
    }
    if (newPassword !== confirmPassword) {
        alert("两次输入的新密码不一致");
        return;
    }
}
```

```

$.ajax({
  url: "/volunteer/password/change",
  type: "POST",
  data: {oldPassword: oldPassword, newPassword: newPassword},
  success: function (result) {
    if (!result || result.code !== 100) {
      alert((result && result.msg) ? result.msg : "密码修改失败，请稍后重试");
      return;
    }
    $("#old_password").val("");
    $("#new_password").val("");
    $("#confirm_password").val("");
    alert("密码已修改，请重新登录");
    window.location.href = "/logout";
  },
  error: function (result) {
    console.log(result);
    alert("网络异常，请稍后重试");
  }
});
});
}

$(document).ready(function () {
  loadProfile();
});
</script>
</body>
</html>

```

```

=====
==
源码文件 177: src/main/resources/templates/volunteer/show.html (前端页面)
=====
==

```

```

<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="icon" href="/console/assets/images/favicon.png" type="image/x-icon">
  <link rel="shortcut icon" href="/console/assets/images/favicon.png" type="image/x-icon">
  <title>志愿秀 - E志愿志愿者服务平台 V1.0</title>

  <link rel="stylesheet" type="text/css" href="/console/assets/css/fontawesome.css">

  <link rel="stylesheet" type="text/css" href="/console/assets/css/icofont.css">

  <link rel="stylesheet" type="text/css" href="/console/assets/css/themify.css">

  <link rel="stylesheet" type="text/css" href="/console/assets/css/flag-icon.css">

```

```

<link rel="stylesheet" type="text/css" href="/console/assets/css/feather-icon.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/bootstrap.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/style.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/responsive.css">
</head>
<body>

<div class="loader-wrapper">
  <div class="theme-loader">
    <div class="loader-p"></div>
  </div>
</div>

<div class="page-wrapper compact-wrapper" id="pageWrapper">
  <div class="page-main-header">
    <div class="main-header-right row m-0">
      <div class="main-header-left">
        <div class="logo-wrapper"><a href="index.html"><img class="img-fluid"
s/logo/app-logo.svg" alt=""></a>
          </div>
          <div class="dark-logo-wrapper"><a href="index.html"><img class="img-fluid"
images/logo/app-logo-dark.svg"
          alt=""></a></div>
        <div class="toggle-sidebar"><i class="status_toggle middle" data-feather="side
bar"
          id="sidebar-toggle"></i></div>
      </div>
      <div class="left-menu-header col">
        <h6 class="f-w-600 mb-0">志愿者工作台</h6>
      </div>
      <div class="nav-right col pull-right right-menu p-0">
        <ul class="nav-menus">
          <li><a class="text-dark" href="#" onclick="javascript:toggleFullScreen
()><i
            data-feather="maximize"></i></a></li>
          <li class="p-0">
            <button class="btn btn-primary-light" type="button"><a href="/logout">
<i
              data-feather="power"></i>退出登录</a></button>
          </li>
        </ul>
      </div>
      <div class="d-lg-none mobile-toggle pull-right w-auto"><i data-feather="menu"></i>
</div>
    </div>
  </div>

  <div class="page-body-wrapper sidebar-icon">
    <header class="main-nav">

```

```

        <div class="sidebar-user text-center">
            <a class="setting-primary" href="profile.html"><i data-feather="settings"></i>
</a>
            
            <a href="profile.html"><h6 class="mt-3 f-14 f-w-600" id="sidebar_volunteer_nam
e">志愿者</h6></a>
            <p class="mb-0 font-roboto" id="sidebar_volunteer_id">编号: —</p>
        </div>
        <nav>
            <div class="main-navbar">
                <div class="left-arrow" id="left-arrow"><i data-feather="rewind"></i></div>
            >
                <div id="mainnav">
                    <ul class="nav-menu custom-scrollbar">
                        <li class="back-btn">
                            <div class="mobile-back text-end"><span>返回</span><i class="f
a fa-angle-right ps-2"
                                aria-hidden
                                ="true"></i></div>
                        </li>
                        <li class="sidebar-main-title">
                            <div><h6>志愿服务</h6></div>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="index.ht
ml"><i
                                data-feather="home"></i><span>个人工作台</span></a>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="activity
_plaza.html"><i
                                data-feather="compass"></i><span>活动广场</span></a>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="my_parti
cipation.html"><i
                                data-feather="clipboard"></i><span>我的报名</span></a>
                        </li>
                        <li class="sidebar-main-title">
                            <div><h6>我的中心</h6></div>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="profile.
html"><i
                                data-feather="user"></i><span>我的档案</span></a>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="notifica
tions.html"><i
                                data-feather="bell"></i><span>消息通知</span></a>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="trainin
g.html"><i
                                data-feather="book-open"></i><span>培训与公告</span></a>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title active" href="s
how.html"><i
                                data-feather="image"></i><span>志愿秀</span></a>
                        </li>
                        <li class="dropdown"><a class="nav-link menu-title" href="certific
ate.html"><i
                                data-feather="award"></i><span>志愿服务证明</span></a>
                        </li>
                    </ul>
                </div>
                <div class="right-arrow" id="right-arrow"><i data-feather="play"></i></div>
            >
        </div>
    </nav>

```

```

</header>

<div class="page-body">

    <div class="container-fluid">
        <div class="row">
            <div class="col-xl-12">
                <div class="card">
                    <div class="card-header">
                        <h5>发布志愿秀</h5>
                    </div>
                    <div class="card-body">
                        <p class="text-muted small mb-2">分享您参与志愿服务的所见所感，可关
联本人已通过报名的活动并上传现场图片。</p>
                        <div class="form-group">
                            <label class="col-form-label" for="show_detail">分享内容</l
abel>

                            <textarea class="form-control" id="show_detail" rows="4"
                                placeholder="请输入本次志愿服务的分享内容"></textare
a>

                        </div>
                        <div class="row">
                            <div class="col-md-6">
                                <div class="form-group">
                                    <label class="col-form-label" for="show_activity">

                                        <select class="form-control" id="show_activity">
                                            <option value="">不关联活动</option>
                                        </select>
                                    </div>
                                </div>
                                <div class="col-md-6">
                                    <div class="form-group">
                                        <label class="col-form-label" for="show_pictures">

                                            <input class="form-control" type="file" id="show_p
ictures" accept="image/*"

                                                multiple>
                                            </div>
                                        </div>
                                    </div>
                                <div class="btn btn-primary" type="button" id="btn_publish_
show"
                                    onclick="publishShow()">发布志愿秀
                                </div>
                            </div>
                        </div>
                    </div>
                <div class="col-xl-12">
                    <div class="card">
                        <div class="card-header">
                            <h5 class="float-start">我的志愿秀</h5>
                            <a class="btn btn-outline-primary btn-sm float-end" href="/por
tal/shows.html">

                                去志愿秀广场
                            </a>
                        </div>
                        <div class="card-body">

```

```

    <p class="text-muted small mb-2">仅展示本人发布的志愿秀，浏览数与点  
赞数由门户志愿秀广场的访问与点赞累计。</p>  
    <div class="table-responsive">  
      <table class="table table-striped table-bordered">  
        <thead>  
          <tr>  
            <th scope="col">分享内容</th>  
            <th scope="col">分享时间</th>  
            <th scope="col">图片数量</th>  
            <th scope="col">浏览数</th>  
            <th scope="col">点赞数</th>  
          </tr>  
        </thead>  
        <tbody id="show_table">  
          <tr>  
            <td colspan="5" class="text-center text-muted">数据加载中.....</td>  
          </tr>  
        </tbody>  
      </table>  
    </div>  
    <div class="row mt-3">  
      <div class="col-md-6">  
        <span class="text-muted small" id="show_total">共 0 条</span>  
  
      </div>  
      <div class="col-md-6">  
        <nav class="float-end">  
          <ul class="pagination pagination-primary mb-0">  
            <li class="page-item" id="show_prev"><a class="page-link" href="#">上一页</a></li>  
            <li class="page-item disabled"><a class="page-link" href="#">第 1 / 1 页</a></li>  
            <li class="page-item" id="show_next"><a class="page-link" href="#">下一页</a></li>  
          </ul>  
        </nav>  
      </div>  
    </div>  
  </div>  
</div>  
<script src="/console/assets/js/jquery-3.5.1.min.js"></script>  
<script src="/console/assets/js/icons/feather-icon/feather.min.js"></script>  
<script src="/console/assets/js/icons/feather-icon/feather-icon.js"></script>
```

```

<script src="/console/assets/js/sidebar-menu.js"></script>
<script src="/console/assets/js/config.js"></script>

<script src="/console/assets/js/bootstrap/popper.min.js"></script>
<script src="/console/assets/js/bootstrap/bootstrap.min.js"></script>

<script src="/console/assets/js/script.js"></script>

<script type="text/javascript">

    function formatTime(value) {
        if (!value) {
            return "-";
        }
        return String(value).replace("T", " ").substring(0, 16);
    }

    var showPage = 1;
    var showSize = 10;
    var showTotal = 0;

    function loadProfile() {
        $.ajax({
            url: "/volunteer/profile",
            type: "GET",
            success: function (result) {
                if (!result || result.code !== 100 || !result.extend || !result.extend.profil
e) {
                    if (result && result.msg) {
                        alert(result.msg);
                    }
                    return;
                }
                var profile = result.extend.profile;
                $("#sidebar_volunteer_name").text(profile.volunteerName || "志愿者");
                $("#sidebar_volunteer_id").text("编号: " + (profile.volunteerId || "-"));
            },
            error: function (result) {
                console.log(result);
            }
        });
    }

    function loadActivityOptions() {
        $.ajax({
            url: "/volunteer/myParticipations",
            type: "GET",
            data: {page: 1, size: 100},
            success: function (result) {
                if (!result || result.code !== 100 || !result.extend) {

```

```

        if (result && result.msg) {
            alert(result.msg);
        }
        return;
    }
    var $select = $("#show_activity");
    $select.empty();
    $select.append($("#<option></option>").val("").text("不关联活动"));
    $.each(result.extend.list || [], function (index, item) {
        if (String(item.participateApplystate) !== "1") {
            return;
        }
        $select.append($("#<option></option>").val(item.activityNum).text(item.acti
vityName));
    });
    },
    error: function (result) {
        console.log(result);
    }
    });
}

function renderShows(shows) {
    var html = "";
    if (!shows || shows.length === 0) {
        html = '<tr><td colspan="5" class="text-center text-muted">暂无数据</td></tr>';
    } else {
        $.each(shows, function (index, item) {
            var pictureCount = (item.pictureCount === null || item.pictureCount === undefi
ned)
                ? "-" : item.pictureCount + " 张";
            var browseCount = Number(item.showBrowse);
            if (isNaN(browseCount) || browseCount < 0) {
                browseCount = 0;
            }
            var likeCount = Number(item.showLike);
            if (isNaN(likeCount) || likeCount < 0) {
                likeCount = 0;
            }
            html += "<tr>"
                + "<td>" + (item.showDetail || "-") + "</td>"
                + "<td>" + formatTime(item.showSharetime) + "</td>"
                + "<td>" + pictureCount + "</td>"
                + "<td>" + browseCount + " 次</td>"
                + "<td>" + likeCount + " 次</td>"
                + "</tr>";
        });
    }
    $("#show_table").html(html);
}

function renderPager(total, current, pageSize) {

```

```
    showTotal = Number(total);
    if (isNaN(showTotal) || showTotal < 0) {
        showTotal = 0;
    }
    var currentPage = Number(current);
    if (isNaN(currentPage) || currentPage < 1) {
        currentPage = 1;
    }
    var currentSize = Number(pageSize);
    if (isNaN(currentSize) || currentSize < 1) {
        currentSize = showSize;
    }
    var totalPages = showTotal > 0 ? Math.ceil(showTotal / currentSize) : 1;
    $("#show_total").text("共 " + showTotal + " 条");
    $("#show_page_info").text("第 " + currentPage + " / " + totalPages + " 页");
    $("#show_prev").toggleClass("disabled", currentPage <= 1);
    $("#show_next").toggleClass("disabled", showTotal === 0 || currentPage >= totalPages);
}

function loadShows() {
    $.ajax({
        url: "/volunteer/content/shows",
        type: "GET",
        data: {page: showPage, size: showSize},
        success: function (result) {
            if (!result || result.code !== 100 || !result.extend) {
                if (result && result.msg) {
                    alert(result.msg);
                }
                return;
            }
            renderShows(result.extend.list);
            renderPager(result.extend.total, result.extend.page, result.extend.size);
        },
        error: function (result) {
            console.log(result);
        }
    });
}

function changeShowPage(step) {
    var nextPage = showPage + step;
    var totalPages = showTotal > 0 ? Math.ceil(showTotal / showSize) : 1;
    if (nextPage < 1 || nextPage > totalPages) {
        return;
    }
    showPage = nextPage;
    loadShows();
}
```

```

function resetShowForm() {
    $("#show_detail").val("");
    $("#show_activity").val("");
    $("#show_pictures").val("");
}

function publishShow() {
    var detail = $.trim($("#show_detail").val());
    if (!detail) {
        alert("请输入分享内容");
        return;
    }
    var activityNum = $("#show_activity").val();
    var fileInput = document.getElementById("show_pictures");
    var files = (fileInput && fileInput.files) ? fileInput.files : [];
    $("#btn_publish_show").prop("disabled", true);
    $.ajax({
        url: "/volunteer/content/show/publish",
        type: "POST",
        data: {detail: detail, activityNum: activityNum},
        success: function (result) {
            if (!result || result.code !== 100 || !result.extend || !result.extend.showNum
m) {
                $("#btn_publish_show").prop("disabled", false);
                alert((result && result.msg) ? result.msg : "志愿秀发布失败, 请稍后重试");
                return;
            }
            var msg = result.msg ? result.msg : "志愿秀发布成功";
            if (files.length === 0) {
                finishPublish(msg);
                return;
            }
            uploadPicture(result.extend.showNum, files, 0, msg);
        },
        error: function (result) {
            console.log(result);
            $("#btn_publish_show").prop("disabled", false);
            alert("网络异常, 请稍后重试");
        }
    });
}

function uploadPicture(showNum, files, index, publishMsg) {
    if (index >= files.length) {
        finishPublish(publishMsg);
        return;
    }
    var formData = new FormData();
    formData.append("showNum", showNum);
    formData.append("file", files[index]);
    $.ajax({

```

```

        url: "/volunteer/content/show/picture",
        type: "POST",
        data: formData,
        processData: false,
        contentType: false,
        success: function (result) {
            if (!result || result.code !== 100) {
                var failMsg = (result && result.msg) ? result.msg : "图片上传失败, 请稍后重
试";
                finishPublish(publishMsg + "; " + failMsg);
                return;
            }
            uploadPicture(showNum, files, index + 1, result.msg ? result.msg : publishMs
g);
        },
        error: function (result) {
            console.log(result);
            finishPublish(publishMsg + "; 图片上传失败, 请稍后重试");
        }
    });
}

function finishPublish(msg) {
    $("#btn_publish_show").prop("disabled", false);
    alert(msg);
    resetShowForm();
    showPage = 1;
    loadShows();
}

$(document).ready(function () {
    loadProfile();
    loadActivityOptions();
    loadShows();
});
</script>
</body>
</html>

```

```

=====
==
源码文件 178: src/main/resources/templates/volunteer/training.html (前端页面)
=====

```

```

==
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="icon" href="/console/assets/images/favicon.png" type="image/x-icon">
    <link rel="shortcut icon" href="/console/assets/images/favicon.png" type="image/x-icon">
    <title>培训与公告 - E志愿志愿者服务平台 V1.0</title>

```

```

<link rel="stylesheet" type="text/css" href="/console/assets/css/fontawesome.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/icofont.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/themify.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/flag-icon.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/feather-icon.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/bootstrap.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/style.css">
<link rel="stylesheet" type="text/css" href="/console/assets/css/responsive.css">
</head>
<body>

<div class="loader-wrapper">
  <div class="theme-loader">
    <div class="loader-p"></div>
  </div>
</div>

<div class="page-wrapper compact-wrapper" id="pageWrapper">

  <div class="page-main-header">
    <div class="main-header-right row m-0">
      <div class="main-header-left">
        <div class="logo-wrapper"><a href="index.html"><img class="img-fluid"
s/logo/app-logo.svg" alt=""></a>
          </div>
          <div class="dark-logo-wrapper"><a href="index.html"><img class="img-fluid"
images/logo/app-logo-dark.svg"
          alt=""></a></div>
        <div class="toggle-sidebar"><i class="status_toggle middle" data-feather="side
bar"
          id="sidebar-toggle"></i></div>
      </div>
      <div class="left-menu-header col">
        <h6 class="f-w-600 mb-0">志愿者工作台</h6>
      </div>
      <div class="nav-right col pull-right right-menu p-0">
        <ul class="nav-menus">
          <li><a class="text-dark" href="#" onclick="javascript:toggleFullScreen
() "><i
            data-feather="maximize"></i></a></li>
          <li class="p-0">
            <button class="btn btn-primary-light" type="button"><a href="/logout">
<i
              data-feather="power"></i>退出登录</a></button>
          </li>
        </ul>

```

```

        </div>
        <div class="d-lg-none mobile-toggle pull-right w-auto"><i data-feather="menu"></i>
</div>
    </div>
</div>

<div class="page-body-wrapper sidebar-icon">
    <header class="main-nav">
        <div class="sidebar-user text-center">
            <a class="setting-primary" href="profile.html"><i data-feather="settings"></i>
</a>
            
            <a href="profile.html"><h6 class="mt-3 f-14 f-w-600" id="sidebar_volunteer_name">志愿者</h6></a>
            <p class="mb-0 font-roboto" id="sidebar_volunteer_id">编号: -</p>
        </div>
        <nav>
            <div class="main-navbar">
                <div class="left-arrow" id="left-arrow"><i data-feather="rewind"></i></div>
            >
            <div id="mainnav">
                <ul class="nav-menu custom-scrollbar">
                    <li class="back-btn">
                        <div class="mobile-back text-end"><span>返回</span><i class="fa fa-angle-right ps-2"
                        aria-hidden="true"></i></div>
                    </li>
                    <li class="sidebar-main-title">
                        <div><h6>志愿服务</h6></div>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title" href="index.html"><i
                        data-feather="home"></i><span>个人工作台</span></a>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title" href="activity_plaza.html"><i
                        data-feather="compass"></i><span>活动广场</span></a>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title" href="my_participation.html"><i
                        data-feather="clipboard"></i><span>我的报名</span></a>
                    </li>
                    <li class="sidebar-main-title">
                        <div><h6>我的中心</h6></div>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title" href="profile.html"><i
                        data-feather="user"></i><span>我的档案</span></a>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title" href="notifications.html"><i
                        data-feather="bell"></i><span>消息通知</span></a>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title active" href="training_raining.html"><i
                        data-feather="book-open"></i><span>培训与公告</span></a>
                    </li>
                    <li class="dropdown"><a class="nav-link menu-title" href="show.html"><i
                        data-feather="image"></i><span>志愿秀</span></a>
                    </li>
                </ul>
            </div>
        </nav>
    </div>
</div>

```

```

        <li class="dropdown"><a class="nav-link menu-title" href="certific
ate.html"><i
            data-feather="award"></i><span>志愿服务证明</span></a>
        </li>
    </ul>
</div>
<div class="right-arrow" id="right-arrow"><i data-feather="play"></i></div>
>
    </div>
</nav>
</header>

<div class="page-body">
    <div class="container-fluid">
        <div class="row">
            <div class="col-xl-12">
                <div class="card">
                    <div class="card-header">
                        <h5>我的培训</h5>
                    </div>
                    <div class="card-body">
                        <p class="text-muted small mb-2">仅展示本人已通过报名的志愿活动对应
的
培训安排。</p>
                        <div class="table-responsive">
                            <table class="table table-striped table-bordernone">
                                <thead>
                                    <tr>
                                        <th scope="col">培训名称</th>
                                        <th scope="col">培训详情</th>
                                        <th scope="col">开始时间</th>
                                        <th scope="col">结束时间</th>
                                        <th scope="col">培训地点</th>
                                        <th scope="col">关联活动</th>
                                    </tr>
                                </thead>
                                <tbody id="training_table">
                                    <tr>
                                        <td colspan="6" class="text-center text-muted">数
据加载中.....</td>
                                    </tr>
                                </tbody>
                            </table>
                        </div>
                    </div>
                </div>
            </div>
            <div class="col-md-6">
                <span class="text-muted small" id="train_total">共 0
条</span>
            </div>
            <div class="col-md-6">
                <nav class="float-end">
                    <ul class="pagination pagination-primary mb-0">
                        <li class="page-item" id="train_prev"><a class
                            = "page-link" href="#"
                            onclick="changeTrainPage(-1);return fa
lse;">上一页</a></li>
                        <li class="page-item disabled"><a class="page-
link" href="#"

```

```

                                id="train_page_info">第 1 / 1 页</a></
li>
                                <li class="page-item" id="train_next"><a class
="page-link" href="#"
                                onclick="changeTrainPage(1);return fal
se;">下一页</a></li>
                                </ul>
                                </nav>
                                </div>
                                </div>
                                </div>
                                </div>
                                </div>
                                <div class="col-xl-12">
                                <div class="card">
                                <div class="card-header">
                                <h5>活动公告</h5>
                                </div>
                                <div class="card-body">
                                <p class="text-muted small mb-2">仅展示本人已报名志愿活动的公告信
息。</p>
                                <div class="table-responsive">
                                <table class="table table-striped table-bordernone">
                                <thead>
                                <tr>
                                <th scope="col">公告标题</th>
                                <th scope="col">公告内容</th>
                                <th scope="col">关联活动</th>
                                </tr>
                                </thead>
                                <tbody id="announcement_table">
                                <tr>
                                <td colspan="3" class="text-center text-muted">数
据加载中.....</td>
                                </tr>
                                </tbody>
                                </table>
                                </div>
                                <div class="row mt-3">
                                <div class="col-md-6">
                                <span class="text-muted small" id="anno_total">共 0 条
</span>
                                </div>
                                <div class="col-md-6">
                                <nav class="float-end">
                                <ul class="pagination pagination-primary mb-0">
                                <li class="page-item" id="anno_prev"><a class
="page-link" href="#"
                                onclick="changeAnnoPage(-1);return fal
se;">上一页</a></li>
                                <li class="page-item disabled"><a class="page-
link" href="#"
                                id="anno_page_info">第 1 / 1 页</a></li>
                                <li class="page-item" id="anno_next"><a class
="page-link" href="#"
                                onclick="changeAnnoPage(1);return fals
e;">下一页</a></li>
                                </ul>
                                </nav>
                                </div>
                                </div>

```

```
        </div>
      </div>
    </div>
  </div>
</div>

<script src="/console/assets/js/jquery-3.5.1.min.js"></script>

<script src="/console/assets/js/icons/feather-icon/feather.min.js"></script>
<script src="/console/assets/js/icons/feather-icon/feather-icon.js"></script>

<script src="/console/assets/js/sidebar-menu.js"></script>
<script src="/console/assets/js/config.js"></script>

<script src="/console/assets/js/bootstrap/popper.min.js"></script>
<script src="/console/assets/js/bootstrap/bootstrap.min.js"></script>

<script src="/console/assets/js/script.js"></script>

<script type="text/javascript">

  function formatTime(value) {
    if (!value) {
      return "-";
    }
    return String(value).replace("T", " ").substring(0, 16);
  }

  function formatTrainingTime(training) {
    var begin = formatTime(training.trainingBeginTime);
    var end = formatTime(training.trainingEndTime);
    if (begin === "-" && end === "-") {
      return "-";
    }
    return begin + " ~ " + end;
  }

  var trainPage = 1;
  var trainSize = 10;
  var trainTotal = 0;
  var annoPage = 1;
  var annoSize = 10;
  var annoTotal = 0;

  function loadProfile() {
    $.ajax({
```

```

        url: "/volunteer/profile",
        type: "GET",
        success: function (result) {
            if (!result || result.code !== 100 || !result.extend || !result.extend.profil
e) {
                if (result && result.msg) {
                    alert(result.msg);
                }
                return;
            }
            var profile = result.extend.profile;
            $("#sidebar_volunteer_name").text(profile.volunteerName || "志愿者");
            $("#sidebar_volunteer_id").text("编号: " + (profile.volunteerId || "-"));
        },
        error: function (result) {
            console.log(result);
        }
    });
}

function renderPager(prefix, total, current, pageSize, defaultSize) {
    var totalCount = Number(total);
    if (isNaN(totalCount) || totalCount < 0) {
        totalCount = 0;
    }
    var currentPage = Number(current);
    if (isNaN(currentPage) || currentPage < 1) {
        currentPage = 1;
    }
    var currentSize = Number(pageSize);
    if (isNaN(currentSize) || currentSize < 1) {
        currentSize = defaultSize;
    }
    var totalPages = totalCount > 0 ? Math.ceil(totalCount / currentSize) : 1;
    $("##" + prefix + "_total").text("共 " + totalCount + " 条");
    $("##" + prefix + "_page_info").text("第 " + currentPage + " / " + totalPages + " 页");
    $("##" + prefix + "_prev").toggleClass("disabled", currentPage <= 1);
    $("##" + prefix + "_next").toggleClass("disabled", totalCount === 0 || currentPage >= t
otalPages);
    return totalCount;
}

function renderTrainings(trainings) {
    var html = "";
    if (!trainings || trainings.length === 0) {
        html = '<tr><td colspan="6" class="text-center text-muted">暂无数据</td></tr>';
    } else {
        $.each(trainings, function (index, item) {
            html += "<tr>"
                + "<td>" + (item.trainingName || "-") + "</td>"
                + "<td>" + (item.trainingDetail || "-") + "</td>"
                + "<td>" + formatTime(item.trainingBegintime) + "</td>"

```

```

        + "<td>" + formatTime(item.trainingEndtime) + "</td>"
        + "<td>" + (item.trainingLocation || "-") + "</td>"
        + "<td>" + (item.activityName || "-") + "</td>"
        + "</tr>";
    });
}
$("#training_table").html(html);
}

function loadTrainings() {
    $.ajax({
        url: "/volunteer/content/trainings",
        type: "GET",
        data: {page: trainPage, size: trainSize},
        success: function (result) {
            if (!result || result.code !== 100 || !result.extend) {
                if (result && result.msg) {
                    alert(result.msg);
                }
                return;
            }
            renderTrainings(result.extend.list);
            trainTotal = renderPager("train", result.extend.total, result.extend.page,
                result.extend.size, trainSize);
        },
        error: function (result) {
            console.log(result);
        }
    });
}

function changeTrainPage(step) {
    var nextPage = trainPage + step;
    var totalPages = trainTotal > 0 ? Math.ceil(trainTotal / trainSize) : 1;
    if (nextPage < 1 || nextPage > totalPages) {
        return;
    }
    trainPage = nextPage;
    loadTrainings();
}

function renderAnnouncements(announcements) {
    var html = "";
    if (!announcements || announcements.length === 0) {
        html = '<tr><td colspan="3" class="text-center text-muted">暂无数据</td></tr>';
    } else {
        $.each(announcements, function (index, item) {
            html += "<tr>"
                + "<td>" + (item.actannouncementName || "-") + "</td>"
                + "<td>" + (item.actannouncementDetail || "-") + "</td>"

```

```

        + "<td>" + (item.activityName || "-") + "</td>"
        + "</tr>";
    });
}
$("#announcement_table").html(html);
}

function loadAnnouncements() {
    $.ajax({
        url: "/volunteer/content/announcements",
        type: "GET",
        data: {page: annoPage, size: annoSize},
        success: function (result) {
            if (!result || result.code !== 100 || !result.extend) {
                if (result && result.msg) {
                    alert(result.msg);
                }
                return;
            }
            renderAnnouncements(result.extend.list);
            annoTotal = renderPager("anno", result.extend.total, result.extend.page,
                result.extend.size, annoSize);
        },
        error: function (result) {
            console.log(result);
        }
    });
}

function changeAnnoPage(step) {
    var nextPage = annoPage + step;
    var totalPages = annoTotal > 0 ? Math.ceil(annoTotal / annoSize) : 1;
    if (nextPage < 1 || nextPage > totalPages) {
        return;
    }
    annoPage = nextPage;
    loadAnnouncements();
}

$(document).ready(function () {
    loadProfile();
    loadTrainings();
    loadAnnouncements();
});
</script>
</body>
</html>

```

=====
 ==
 源码文件 179: src/test/java/com/evolunteer/EVolunteerApplicationTests.java (单元测试)

```
=====
==
package com.evoluteer;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class EVoluteerApplicationTests {

    @Test
    @DisplayName("平台上下文可以正常加载")
    void contextLoads() {
    }

}
=====
```

源码文件 180: src/test/java/com/evoluteer/entity/ApiResponseTest.java (单元测试)

```
=====
==
package com.evoluteer.entity;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertSame;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ApiResponseTest {

    @Test
    @DisplayName("成功结果的状态码与提示信息正确")
    void successShouldCarrySuccessCode() {
        ApiResponse response = ApiResponse.success();

        assertEquals(100, response.getCode());
        assertEquals("处理成功!", response.getMsg());
        assertTrue(response.getExtend().isEmpty());
    }

    @Test
    @DisplayName("失败结果的状态码与提示信息正确")
    void failShouldCarryFailCode() {
        ApiResponse response = ApiResponse.fail();

        assertEquals(200, response.getCode());
        assertEquals("处理失败!", response.getMsg());
    }

}
=====
```

```

@Test
@DisplayName("添加业务数据后可被读取且支持连续追加")
void addShouldStoreBusinessData() {
    List<String> activities = List.of("社区爱心助学志愿活动", "敬老院陪伴志愿服务");

    ApiResponse response = ApiResponse.success().add("auditActivity", activities).add("total", activities.size());

    assertEquals(activities, response.getExtend().get("auditActivity"));
    assertEquals(2, response.getExtend().get("total"));
}
}

```

=====
 ==
 源码文件 181: src/test/java/com/evolunteer/enums/VolunteerStarEnumTest.java (单元测试)
 =====

```

package com.evolunteer.enums;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNull;

class VolunteerStarEnumTest {

    @Test
    @DisplayName("按累计服务时长认定星级, 边界值归属上一档")
    void ofShouldMatchStarThresholds() {
        assertEquals("未定级", VolunteerStarEnum.of(0D).getStarName());
        assertEquals("未定级", VolunteerStarEnum.of(99.9).getStarName());
        assertEquals("一星志愿者", VolunteerStarEnum.of(100D).getStarName());
        assertEquals("二星志愿者", VolunteerStarEnum.of(300D).getStarName());
        assertEquals("三星志愿者", VolunteerStarEnum.of(600D).getStarName());
        assertEquals("四星志愿者", VolunteerStarEnum.of(1000D).getStarName());
        assertEquals("五星志愿者", VolunteerStarEnum.of(1500D).getStarName());
        assertEquals("五星志愿者", VolunteerStarEnum.of(9999D).getStarName());
        assertEquals("未定级", VolunteerStarEnum.of(null).getStarName());
    }

    @Test
    @DisplayName("计算距离下一星级还需的累计服务时长")
    void hoursToNextStarShouldReturnRemainingHours() {
        assertEquals(100D, VolunteerStarEnum.hoursToNextStar(0D));
        assertEquals(97D, VolunteerStarEnum.hoursToNextStar(3D), 0.001);
        assertEquals(0D, VolunteerStarEnum.hoursToNextStar(1500D));
    }

    @Test
    @DisplayName("下一星级在已达最高星级时为空")

```

```

    void nextStarShouldBeNullForTopStar() {
        assertEquals(VolunteerStarEnum.ONE_STAR, VolunteerStarEnum.nextStar(99D));
        assertEquals(VolunteerStarEnum.FIVE_STAR, VolunteerStarEnum.nextStar(1200D));
        assertNull(VolunteerStarEnum.nextStar(1500D));
    }
}

```

```

=====
==
源码文件 182: src/test/java/com/evolunteer/utils/CertificateCodeUtilTest.java (单元测试)
=====
==

```

```

package com.evolunteer.utils;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class CertificateCodeUtilTest {

    private static final String SECRET = "evolunteer-test-secret";

    @Test
    @DisplayName("校验码固定为 16 位大写十六进制字符串且可重复计算")
    void codeShouldBeStableHexString() {
        String code = CertificateCodeUtil.code(SECRET, "vol_00001", 3.0, 1);

        assertEquals(16, code.length());
        assertTrue(code.matches("[0-9A-F]{16}"));
        assertEquals(code, CertificateCodeUtil.code(SECRET, "vol_00001", 3.0, 1));
    }

    @Test
    @DisplayName("累计时长或服务记录数变化时校验码随之变化")
    void codeShouldChangeWithServiceData() {
        String code = CertificateCodeUtil.code(SECRET, "vol_00001", 3.0, 1);

        assertNotEquals(code, CertificateCodeUtil.code(SECRET, "vol_00001", 3.5, 1));
        assertNotEquals(code, CertificateCodeUtil.code(SECRET, "vol_00001", 3.0, 2));
        assertNotEquals(code, CertificateCodeUtil.code(SECRET, "vol_00002", 3.0, 1));
    }

    @Test
    @DisplayName("密钥不同时同一份服务数据的校验码不同，第三方无法伪造")
    void codeShouldDependOnSecret() {
        assertNotEquals(CertificateCodeUtil.code(SECRET, "vol_00001", 3.0, 1),
            CertificateCodeUtil.code("another-secret", "vol_00001", 3.0, 1));
    }
}

```

```

@Test
@DisplayName("累计时长为空时按 0 小时计算且不抛出异常")
void codeShouldTolerateNullDuration() {
    String code = CertificateCodeUtil.code(SECRET, "vol_00001", null, null);

    assertEquals(16, code.length());
    assertEquals(code, CertificateCodeUtil.code(SECRET, "vol_00001", 0D, 0));
}
}

=====
==
源码文件 183: src/test/java/com/evolunteer/utils/CsvExporterTest.java (单元测试)
=====
==
package com.evolunteer.utils;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class CsvExporterTest {

    @Test
    @DisplayName("导出内容带 UTF-8 字节序标记并使用回车换行分隔")
    void toCsvShouldWriteBomAndLineSeparator() {
        String csv = CsvExporter.toCsv(Arrays.asList("姓名", "时长"), Collections.singletonList(
            Arrays.asList("张三", "3.0")));

        assertTrue(csv.startsWith("\uFEFF"));
        assertEquals("\uFEFF姓名,时长\r\n张三,3.0\r\n", csv);
    }

    @Test
    @DisplayName("包含逗号、引号与换行的字段按 RFC 4180 规则转义")
    void escapeShouldQuoteSpecialCharacters() {
        assertEquals("普通文本", CsvExporter.escape("普通文本"));
        assertEquals("\"含,逗号\"", CsvExporter.escape("含,逗号"));
        assertEquals("\"含\"\"引号\"", CsvExporter.escape("含\"引号"));
        assertEquals("\"含换行\n文本\"", CsvExporter.escape("含换行\n文本"));
        assertEquals("", CsvExporter.escape(null));
    }

    @Test
    @DisplayName("空数据行只输出表头")
    void toCsvShouldHandleEmptyRows() {
        List<String> headers = Arrays.asList("活动编号", "活动名称");
    }
}

```

```

        assertEquals("\uFEFF活动编号,活动名称\r\n", CsvExporter.toCsv(headers, Collections.emptyList()));
    }
}

```

=====
 源码文件 184: src/test/java/com/evolunteer/utils/DateTimeParserTest.java (单元测试)
 =====

```

package com.evolunteer.utils;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.text.SimpleDateFormat;
import java.util.Date;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNull;
import static org.junit.jupiter.api.Assertions.assertThrows;

class DateTimeParserTest {

    @Test
    @DisplayName("支持浏览器日期时间控件与历史日期格式")
    void parseShouldSupportMultiplePatterns() throws Exception {
        assertEquals("2026-10-18 09:00:00", format(DateTimeParser.parse("2026-10-18T09:00")));
        assertEquals("2026-10-18 09:00:00", format(DateTimeParser.parse("2026-10-18 09:00")));
        assertEquals("2026-10-18 09:00:00", format(DateTimeParser.parse("2026-10-18T09:00:00")));
        assertEquals("2026-10-18 00:00:00", format(DateTimeParser.parse("2026-10-18")));
        assertEquals("2026-10-18 00:00:00", format(DateTimeParser.parse("10/18/2026")));
    }

    @Test
    @DisplayName("空白时间文本解析为空值，由存储过程校验必填")
    void parseShouldReturnNullForBlankText() {
        assertNull(DateTimeParser.parse(null));
        assertNull(DateTimeParser.parse(""));
        assertNull(DateTimeParser.parse("   "));
    }

    @Test
    @DisplayName("非法时间文本抛出可展示给用户的异常")
    void parseShouldRejectInvalidText() {
        IllegalArgumentException exception = assertThrows(IllegalArgumentException.class,
            () -> DateTimeParser.parse("2026年10月18日"));

        assertEquals(true, exception.getMessage().contains("时间格式不正确"));
        assertThrows(IllegalArgumentException.class, () -> DateTimeParser.parse("2026-13-40"));
    }
}

```

```
        private String format(Date date) {
            return new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(date);
        }
    }
}
```

=====
==
源码文件 185: src/test/java/com/evolunteer/utis/MD5UtilTest.java (单元测试)
=====

```
==
package com.evolunteer.utis;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

class MD5UtilTest {

    @Test
    @DisplayName("相同密码的摘要结果一致且为 32 位十六进制字符串")
    void encodeShouldReturnStableHexCode() {
        String encoded = MD5Util.encode("123456");

        assertEquals("e10adc3949ba59abbe56e057f20f883e", encoded);
        assertEquals(32, encoded.length());
        assertTrue(encoded.matches("[0-9a-f]{32}"));
        assertEquals(encoded, MD5Util.encode("123456"));
    }

    @Test
    @DisplayName("不同密码的摘要结果不同")
    void encodeShouldDifferForDifferentPasswords() {
        assertNotEquals(MD5Util.encode("123456"), MD5Util.encode("1234567"));
        assertNotEquals(MD5Util.encode("abc"), MD5Util.encode("中国志愿者"));
    }

    @Test
    @DisplayName("中文密码的摘要结果与字符集无关且长度固定")
    void encodeShouldSupportChinesePassword() {
        String encoded = MD5Util.encode("志愿者服务平台");

        assertEquals(32, encoded.length());
        assertEquals(encoded, MD5Util.encode("志愿者服务平台"));
    }

    @Test
    @DisplayName("空明文在计算摘要前由调用方校验")
    void encodeShouldRejectNullPassword() {

```

```

        assertThrows(NullPointerException.class, () -> MD5Util.encode(null));
    }
}

```

=====
 ==
 源码文件 186: src/test/java/com/evolunteer/utils/MatchCalculatorTest.java (单元测试)
 =====

```

==
package com.evolunteer.utils;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class MatchCalculatorTest {

    private static final List<String> ACTIVITY_SKILLS = Arrays.asList("社区服务", "文化宣传");

    @Test
    @DisplayName("技能全部命中且距离在 1 公里内、时间不冲突、有历史参与时匹配度为满分")
    void calculateShouldReturnFullScoreForPerfectMatch() {
        MatchCalculator.MatchResult result = MatchCalculator.calculate(
            Arrays.asList("社区服务", "文化宣传"), ACTIVITY_SKILLS, 500,
            Collections.emptyList(), 2);

        assertEquals(100, result.getScore());
        assertEquals(Arrays.asList("社区服务", "文化宣传"), result.getMatchedSkills());
        assertTrue(result.getReasons().contains("技能匹配: 社区服务、文化宣传 (命中 2/2 项)"));
        assertTrue(result.getReasons().contains("距您 500 米"));
        assertTrue(result.getReasons().contains("与您已报名的活动时间不冲突"));
        assertTrue(result.getReasons().contains("您已参与该组织 2 次活动"));
    }

    @Test
    @DisplayName("技能部分命中时按命中比例折算技能维度得分")
    void calculateShouldScoreSkillByHitRatio() {
        MatchCalculator.MatchResult result = MatchCalculator.calculate(
            Collections.singletonList("社区服务"), ACTIVITY_SKILLS, 1500,
            Collections.emptyList(), 0);

        assertEquals(69, result.getScore());
        assertEquals(Collections.singletonList("社区服务"), result.getMatchedSkills());
    }

    @Test

```

```

@DisplayName("缺少定位时距离维度按中性分值计算并说明原因")
void calculateShouldUseNeutralScoreWhenDistanceUnknown() {
    MatchCalculator.MatchResult result = MatchCalculator.calculate(
        Collections.emptyList(), Collections.emptyList(), null, Collections.emptyList
    ), 0);

    assertEquals(55, result.getScore());
    assertTrue(result.getReasons().contains("未设置常住定位, 按中性距离分值计算"));
}

@Test
@DisplayName("时间冲突时时间维度不得分, 并给出冲突活动名称")
void calculateShouldPenalizeTimeConflict() {
    MatchCalculator.MatchResult result = MatchCalculator.calculate(
        ACTIVITY_SKILLS, ACTIVITY_SKILLS, 300, Collections.singletonList("社区义诊志愿服
务"), 1);

    assertEquals(80, result.getScore());
    assertTrue(result.getReasons().contains("与您已报名的「社区义诊志愿服务」时间冲突"));
}

@Test
@DisplayName("距离超过 2 千米时距离维度不得分且提示距离较远")
void calculateShouldGiveZeroDistanceScoreWhenTooFar() {
    MatchCalculator.MatchResult result = MatchCalculator.calculate(
        Collections.emptyList(), ACTIVITY_SKILLS, 35000, Collections.emptyList(), 0);

    assertEquals(20, result.getScore());
    assertTrue(result.getReasons().contains("距离较远 (约 35.0 公里)"));
}

@Test
@DisplayName("技能标签为空或未登记技能时提示活动要求技能")
void calculateShouldExplainMissingSkillOverlap() {
    MatchCalculator.MatchResult result = MatchCalculator.calculate(
        Collections.singletonList("信息技术"), ACTIVITY_SKILLS, 2000, Collections.empty
List(), 0);

    assertTrue(result.getMatchedSkills().isEmpty());
    assertTrue(result.getReasons().contains("活动要求技能: 社区服务、文化宣传, 与您的技能标签暂未重
叠"));
}
}

```

```

=====
==
源码文件 187: src/test/java/com/evolunteer/utils/PageSupportTest.java (单元测试)
=====
==

```

```

package com.evolunteer.utils;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.evolunteer.entity.ApiResponse;
import com.evolunteer.entity.AuditLog;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

```

```

import java.util.Arrays;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNull;

class PageSupportTest {

    @Test
    @DisplayName("分页参数为空或非法时使用默认值, 超过上限时按上限截断")
    void ofShouldNormalizePageParams() {
        assertEquals(1, PageSupport.of(null, null).getCurrent());
        assertEquals(10, PageSupport.of(null, null).getSize());
        assertEquals(1, PageSupport.of(0, 0).getCurrent());
        assertEquals(10, PageSupport.of(-3, -1).getSize());
        assertEquals(3, PageSupport.of(3, 20).getCurrent());
        assertEquals(100, PageSupport.of(1, 9999).getSize());
    }

    @Test
    @DisplayName("分页结果按固定字段装入统一响应")
    void toResponseShouldFillStandardFields() {
        Page<AuditLog> page = new Page<>(2, 10, 25);
        page.setRecords(Arrays.asList(new AuditLog(), new AuditLog()));

        ApiResponse response = PageSupport.toResponse(page);
        Map<String, Object> extend = response.getExtend();

        assertEquals(100, response.getCode());
        assertEquals(2, extend.get("list") instanceof java.util.List ? ((java.util.List<?>) extend.get("list")).size() : -1);
        assertEquals(25L, extend.get("total"));
        assertEquals(2L, extend.get("page"));
        assertEquals(10L, extend.get("size"));
    }

    @Test
    @DisplayName("检索关键字去空格, 空串统一为 null")
    void normalizeKeywordShouldTrimAndBlankToNull() {
        assertEquals("助学", PageSupport.normalizeKeyword("  助学  "));
        assertNull(PageSupport.normalizeKeyword(""));
        assertNull(PageSupport.normalizeKeyword("   "));
        assertNull(PageSupport.normalizeKeyword(null));
    }
}

```

```

=====
==
源码文件 188: src/test/java/com/evolunteer/utils/RandomPasswordUtilTest.java (单元测试)
=====
==

```

```

package com.evolunteer.utils;

```

```

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertNotEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class RandomPasswordUtilTest {

    @Test
    @DisplayName("生成的随机密码长度为 8 且只包含允许的字符")
    void randomPasswordShouldMatchAllowedCharacters() {
        for (int i = 0; i < 20; i++) {
            String password = RandomPasswordUtil.randomPassword();

            assertEquals(8, password.length());
            assertTrue(password.matches("[2-9A-HJ-NP-Za-km-z]+"), password);
            assertFalse(password.contains("0"));
            assertFalse(password.contains("O"));
            assertFalse(password.contains("1"));
            assertFalse(password.contains("l"));
        }
    }

    @Test
    @DisplayName("连续生成的密码不相同")
    void randomPasswordShouldBeRandom() {
        assertNotEquals(RandomPasswordUtil.randomPassword(), RandomPasswordUtil.randomPassword());
    }
}

```

```

=====
==
源码文件 189: src/test/java/com/evolunteer/utils/SecurePasswordEncoderTest.java (单元测试)
=====
==

```

```

package com.evolunteer.utils;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertNotEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class SecurePasswordEncoderTest {

    private final SecurePasswordEncoder passwordEncoder = new SecurePasswordEncoder();

```

```
@Test
@DisplayName("加盐摘要可以校验原始密码且密文格式带有算法与迭代次数")
void encodeShouldProduceVerifiableCipher() {
    String encoded = passwordEncoder.encode("123456");

    assertTrue(encoded.startsWith("pbkdf2$120000$"));
    assertTrue(passwordEncoder.matches("123456", encoded));
    assertFalse(passwordEncoder.matches("1234567", encoded));
    assertFalse(passwordEncoder.matches("123456", encoded + "x"));
    assertFalse(passwordEncoder.matches(null, encoded));
    assertFalse(passwordEncoder.matches("123456", null));
}

@Test
@DisplayName("同一密码两次加密得到不同密文，避免撞库时批量识破相同密码")
void encodeShouldUseRandomSalt() {
    String first = passwordEncoder.encode("123456");
    String second = passwordEncoder.encode("123456");

    assertNotEquals(first, second);
    assertTrue(passwordEncoder.matches("123456", first));
    assertTrue(passwordEncoder.matches("123456", second));
}

@Test
@DisplayName("历史 MD5 密文可以校验通过并需要进行升级")
void matchesShouldSupportLegacyMd5Cipher() {
    String legacy = MD5Util.encode("123456");

    assertTrue(passwordEncoder.matches("123456", legacy));
    assertFalse(passwordEncoder.matches("654321", legacy));
    assertTrue(passwordEncoder.upgradeEncoding(legacy));
}

@Test
@DisplayName("加盐密文无需再次升级")
void upgradeEncodingShouldSkipSaltedCipher() {
    assertFalse(passwordEncoder.upgradeEncoding(passwordEncoder.encode("123456")));
    assertFalse(passwordEncoder.upgradeEncoding(null));
}

@Test
@DisplayName("非法密文校验返回 false 而不抛出异常")
void matchesShouldRejectMalformedCipher() {
    assertFalse(passwordEncoder.matches("123456", "pbkdf2$abc$notbase64$notbase64"));
    assertFalse(passwordEncoder.matches("123456", "unknown$format"));
    assertEquals("123456", "123456");
}
}
```